

proyecto

puppy fruit

Eros Verdejo

Isaias Rojas

Ariel Ponce

Profesor: Leonardo Gonzalez Catalan

Asignatura: Programación avanzada

Carrera: Ingeniería en informática

GM-1 Descripción del Juego desde la Perspectiva del Jugador

Narrativa y Objetivo

El juego Perro Frutal pone al jugador en el rol de un perro granjero que debe recolectar frutas que caen desde la parte superior de la pantalla. El objetivo es sobrevivir el mayor tiempo posible sin perder las tres vidas disponibles, acumulando puntos para superar el récord histórico registrado en el dispositivo.

Controles

- Flecha izquierda / flecha derecha: mover al perro recolector horizontalmente.
- ENTER o ESPACIO: iniciar partida o reiniciar desde la pantalla de final.
- M: volver al menú principal desde la pantalla de final.
- ESC: salir del juego desde el menú.

Mecánica de Juego

- El jugador controla un perro recolector (sprite 150×120 px) que se mueve horizontalmente por la parte inferior de la pantalla.
- Las frutas caen desde la parte superior con distintas velocidades y trayectorias según su tipo.
- Recolectar una fruta buena (manzana, plátano o naranja) suma entre 10 y 25 puntos.
- Recolectar una fruta podrida quita una vida y resta 10 puntos (mínimo 0). Si el escudo está activo, lo consume en lugar de quitar la vida.
- Al alcanzar 200 puntos el juego pasa automáticamente al nivel Difícil, donde las frutas caen más rápido y hay mayor proporción de frutas podridas.
- La partida termina cuando las vidas llegan a 0, mostrando la pantalla de final con el puntaje obtenido y el máximo histórico o la pantalla de victoria.

Tipos de Frutas

Frutas normales (trayectoria recta, CaidaNormal):

- Manzana, plátano, naranja: suman entre 10 y 25 puntos al ser recolectadas.

Frutas con power-up (trayectoria zigzag, CaidaZigZag):

- Guinda (10% de probabilidad): activa el power-up Doble Puntaje por 8 segundos.
- Kiwi (7% de probabilidad): activa el power-up Cámara Lenta por 6 segundos, reduciendo la velocidad de caída de todas las frutas al 45%.

- Durian (5% de probabilidad): activa el power-up Escudo por 8 segundos.

Fruta especial (trayectoria zigzag, CaidaZigZag):

- Fruta dorada (2% de probabilidad): suma 50 puntos y recupera una vida (máximo 3 vidas).

Fruta mala (trayectoria rápida, CaidaRapida):

- Fruta podrida (30% de probabilidad): quita una vida y resta 10 puntos.

Power-ups

- Doble puntaje (guinda, 8 s): todos los puntos obtenidos se multiplican por 2 mientras esté activo. Se muestra el tiempo restante en pantalla.
- Cámara lenta (kiwi, 6 s): reduce la velocidad de caída de todas las frutas al 45% de su velocidad normal mediante el método `getFactorVelocidadCaida()`. Se muestra el tiempo restante.
- Escudo (durian, 8 s): protege al jugador de la siguiente fruta podrida. Al absorber el daño el escudo se desactiva inmediatamente. Se muestra el tiempo restante.

Pantallas del Juego

- Menú principal: muestra el nombre del juego, las probabilidades de cada power-up, el puntaje máximo histórico e instrucciones de controles.
- Pantalla de juego: muestra puntos actuales, máximo histórico, vidas como corazones, power-ups activos con tiempo restante, nivel actual e instrucciones.

- `finalAprobaste`: fondo personalizado con imagen 'final_aprobaste.png', puntaje de la partida, puntaje máximo histórico y opciones para reiniciar o volver al menú.
- `finalReprobaste`: fondo personalizado con imagen 'final_reprobaste.png', puntaje de la partida, puntaje máximo histórico y opciones para reiniciar o volver al menú.

GM-2 Análisis desde la Perspectiva de Ingeniería de Software

Juego Base: GameLluvia 2024

El juego original consistía en un recolector de gotas de agua con mecánicas muy simples. Estaba compuesto por solo tres clases propias: `GameLluvia` (controladora principal), `Lluvia` (generación y movimiento de gotas) y `Tarro` (jugador). Toda la lógica de colisiones, efectos y generación estaba embebida directamente en `Lluvia.actualizarMovimiento()`.

- `GameLluvia`: clase principal `ApplicationAdapter`. Instanciaba directamente las texturas y sonidos y los pasaba por constructor. Fondo azul fijo (`ScreenUtils.clear` azul oscuro).
- `Lluvia`: manejaba gotas con dos Arrays paralelos (`rainDropsPos` y `rainDropsType`). El tipo 1 era gota mala y el tipo 2 gota buena. Toda la lógica de efecto estaba en un `if/else` dentro de `actualizarMovimiento()`.
- `Tarro`: representaba al jugador. Tenía vidas, puntos y movimiento. El método `dañar()` decrementaba vidas sin ningún control de límite inferior.

Problemas Identificados en el Juego Base

Problema 1: Sistema de vidas sin límite inferior

El método `dañar()` en `Tarro` ejecutaba `vidas--` sin verificación. Las vidas podían llegar a valores negativos (-1, -2, -3...) indefinidamente sin que el juego terminase, generando un estado inconsistente. No existía condición de fin de partida ni pantalla de final.

Problema 2: Un solo tipo de objeto recolectable

Lluvia solo manejaba gotas buenas y malas diferenciadas por un entero (tipo 1 o tipo 2) en Arrays paralelos. No había estructura para soportar múltiples tipos de objetos con distintos comportamientos y efectos.

Problema 3: Arquitectura monolítica poco escalable

La lógica de qué hace cada tipo de gota al ser recolectada estaba embebida con if/else directamente en Lluvia. Agregar un nuevo tipo de objeto requería modificar directamente esa clase, violando el principio Open/Closed. No existían patrones de diseño ni jerarquías de clases.

Problema 4: Recursos no centralizados

Las texturas y sonidos se instanciaban en GameLluvia.create() y se pasaban por parámetro a los constructores. No había ciclo de vida centralizado ni reutilización de recursos.

Problema 5: Sin progresión ni registro

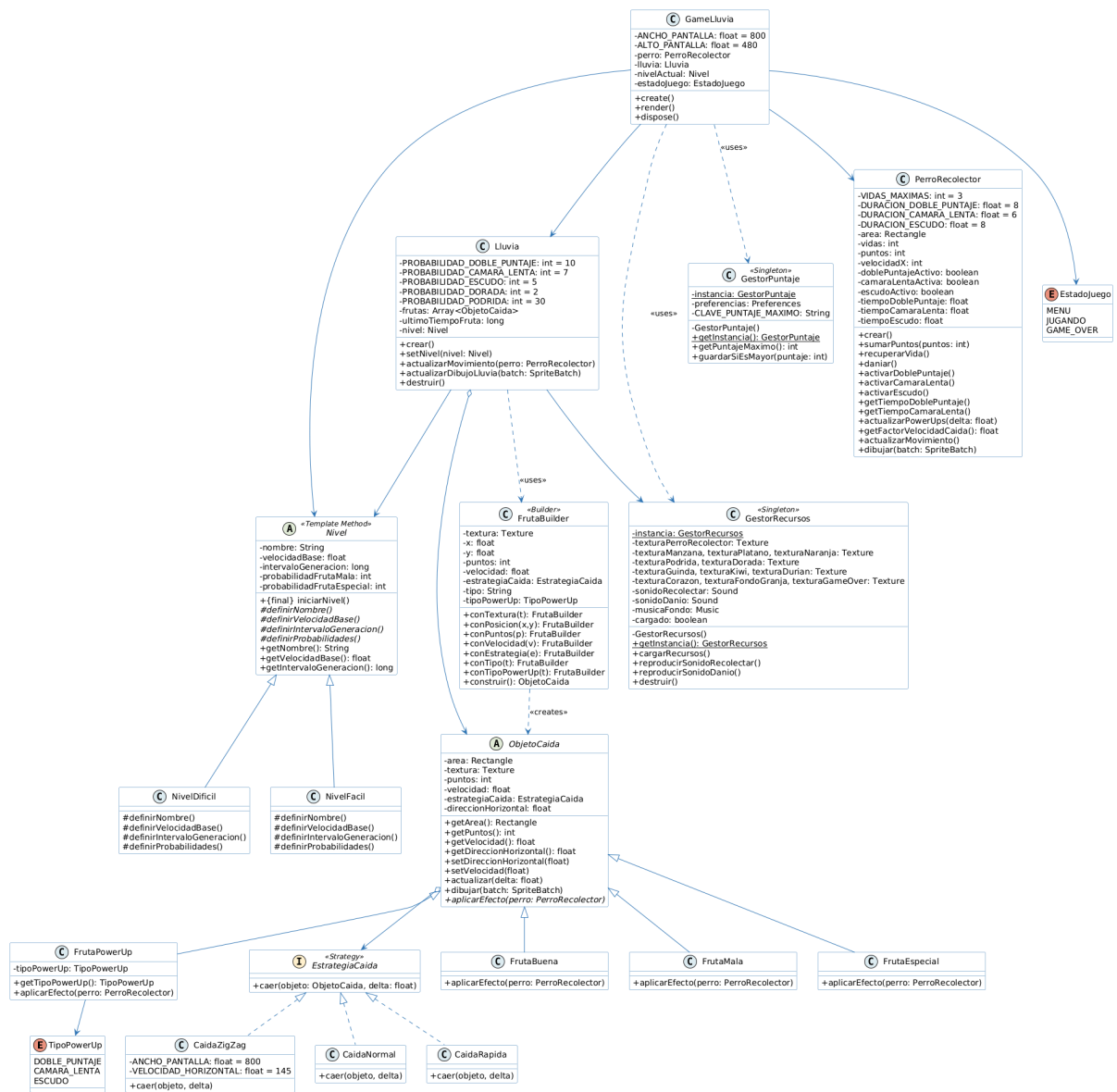
El juego era infinito sin objetivo ni dificultad progresiva. No había sistema de niveles, registro de puntaje máximo, ni pantallas de menú o final.

Modificaciones Implementadas en el Juego Final

- Sistema de vidas corregido: vidas inicia en 3, límite inferior garantizado en 0 mediante doble validación en daniar(). Al llegar a 0 se activa final.
- Jerarquía de objetos: clase abstracta ObjetoCaida con subclases FrutaBuena, FrutaMala, FrutaEspecial y FrutaPowerUp. Cada una implementa su propio aplicarEfecto().
- Sistema de power-ups: enum TipoPowerUp (DOBLE_PUNTAJE, CAMARA_LENTA, ESCUDO) con duración temporal, gestionado por PerroRecolector.actualizarPowerUps().
- Cámara lenta real: Lluvia.actualizarMovimiento() multiplica el delta por perro.getFactorVelocidadCaida(), que devuelve 0.45f cuando el kiwi está activo.

- Estrategias de caída intercambiables: interfaz `EstrategiaCaida` con `CaidaNormal`, `CaidaRapida` y `CaidaZigZag` (con rebote real en bordes usando `direccionHorizontal`).
- Sistema de niveles: clase abstracta `Nivel` con `Template Method`, implementada por `NivelFacil` (vel 260, prob podrida 22%) y `NivelDifícil` (vel 360, prob podrida 34%). Transición automática a los 200 puntos.
- Gestión centralizada de recursos: `GestorRecursos` (Singleton) carga todos los assets incluyendo guinda, kiwi, durián, `perro_recolector`, `final_reprobaste` y música de fondo personalizada.
- Registro de puntaje máximo: `GestorPuntaje` (Singleton) persiste el récord entre sesiones usando `Preferences` de `LibGDX`.
- Personaje rediseñado: `PerroRecolector` reemplaza a `Tarro/Canasta`. Sprite más grande (150×120), gestiona todos los power-ups y sus duraciones.
- Constructor fluido: `FrutaBuilder` con método `conTipoPowerUp()` adicional para construir `FrutaPowerUp`.
- Pantalla final personalizada: fondo con imagen '`final_reprobaste.png`' y visualización del puntaje máximo en grande.
- Pantalla Victoria personalizada: fondo con imagen '`final_aprobaste.png`' y visualización del puntaje máximo en grande.

GM-3 Diagrama UML del Dominio del Juego



[link imagen](#)

Clase Principal: GameLluvia

Clase principal ApplicationAdapter de LibGDX. El ciclo de vida del juego y los estados MENU, JUGANDO y GAME_OVER. Contiene referencias a PerroRecolector, Lluvia y Nivel. Invoca perro.actualizarPowerUps() en cada frame para actualizar los temporizadores de power-ups.

Jugador: PerroRecolector

Reemplaza a Tarro del juego base. Encapsula vidas (máx 3), puntos, estado herido, y los tres power-ups con sus duraciones (doblePuntaje 8s, camaraLenta 6s, escudo 8s). Provee `getFactorVelocidadCaida()` que devuelve 0.45f durante cámara lenta, afectando la velocidad de caída en Lluvia. El sprite tiene dimensiones 150×120 px.

Gestión de Frutas: Lluvia

Gestiona la generación, movimiento, colisiones y dibujo de todas las frutas. Usa `FrutaBuilder` para construir cada `ObjetoCaida`. Aplica el factor de velocidad del perro al delta en `actualizarMovimiento()`, implementando así el efecto de cámara lenta. Las probabilidades están definidas como constantes: podrida 30%, guinda 10%, kiwi 7%, durián 5%, dorada 2%.

Jerarquía de ObjetoCaida

Clase abstracta que define la estructura común: área de colisión (`Rectangle 64×64`), textura, puntos, velocidad, estrategia de caída y dirección horizontal (para `CaidaZigZag`). El método abstracto `aplicarEfecto(PerroRecolector)` es implementado por cada subclase.

- `FrutaBuena`: suma puntos y reproduce sonido de recolección.
- `FrutaMala`: llama `daniar()` en el perro y resta 10 puntos.
- `FrutaEspecial` (fruta dorada): suma 50 puntos y recupera una vida.
- `FrutaPowerUp`: activa el power-up según su `TipoPowerUp` (`DOBLE_PUNTAJE`, `CAMARA_LENTA` o `ESCUDO`).

Enum TipoPowerUp

Enumeración con tres valores: `DOBLE_PUNTAJE`, `CAMARA_LENTA` y `ESCUDO`. `FrutaPowerUp` contiene una instancia de este enum y `FrutaBuilder` la recibe mediante `conTipoPowerUp()`. Permite distinguir el efecto a aplicar sin cadenas de texto ni condicionales adicionales.

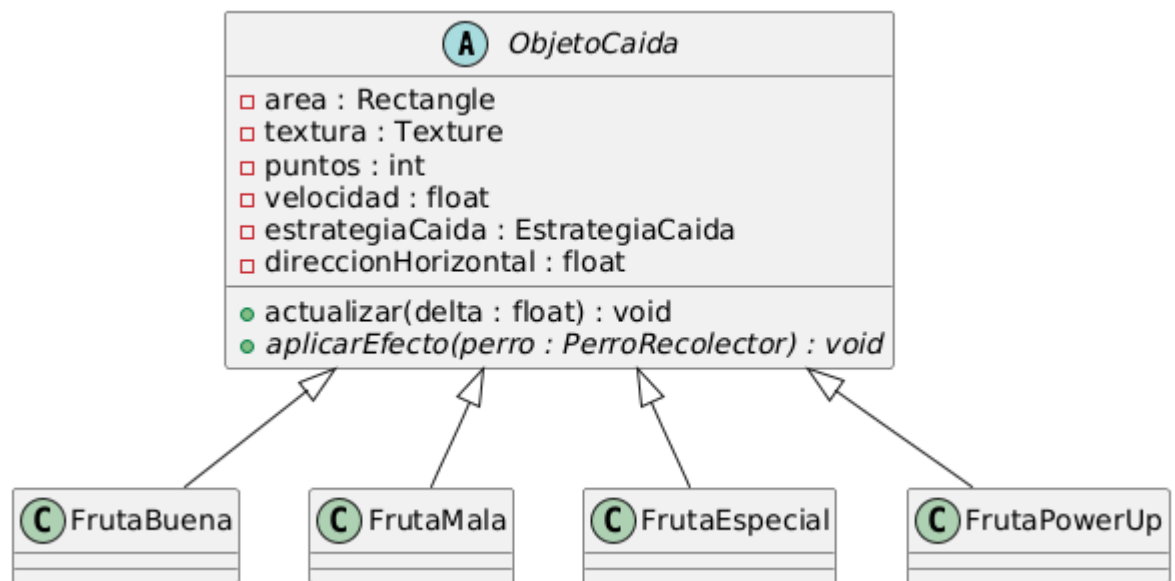
Gestores Singleton

- GestorRecursos: carga y provee todas las texturas (perro, frutas, corazón, fondo granja, final), sonidos y música. Incluye métodos reproducirSonidoRecolectar() y reproducirSonidoDanio() con volumen 0.75f..
- GestorPuntaje: persiste el puntaje máximo en Preferences bajo la clave 'PerroRecolectorFrutaPuntajes'.

GM-4 Clase Abstracta e Interfaz

Clase Abstracta 1: ObjetoCaida

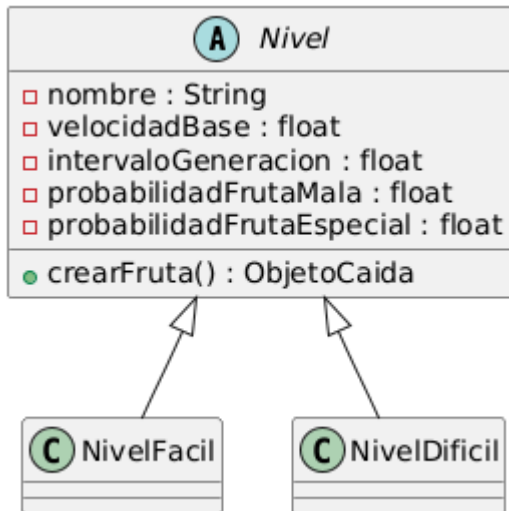
ObjetoCaida es la clase abstracta principal del dominio. Es padre de FrutaBuena, FrutaMala, FrutaEspecial y FrutaPowerUp, y es utilizada activamente en Lluvia mediante Array<ObjetoCaida>. Define todos los atributos comunes y delega el comportamiento de caída en la estrategia.



La dirección horizontal se inicializa aleatoriamente (1f o -1f) en el constructor y es modificada por `CaidaZigZag` al rebotar en los bordes, lo que produce el movimiento zigzag real con rebote.

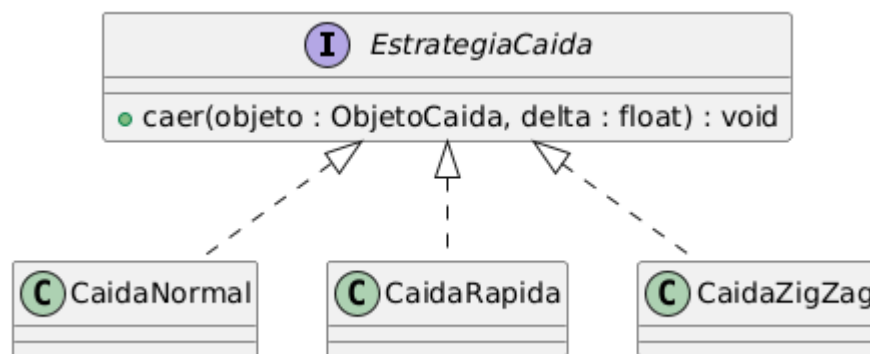
Clase Abstracta 2: Nivel

Nivel es la segunda clase abstracta del proyecto, usada para el patrón Template Method. Es padre de NivelFacil y NivelDificil, y es referenciada por Lluvia y GameLluvia de forma polimórfica. Define los parámetros de dificultad: nombre, velocidadBase, intervaloGeneracion, probabilidadFrutaMala y probabilidadFrutaEspecial.



Interfaz: EstrategiaCaida

Interfaz del patrón Strategy, implementada por CaidaNormal, CaidaRapida y CaidaZigZag. Es utilizada como campo de ObjetoCaida.



- CaidaNormal: `objeto.getArea().y -= velocidad * delta`. Frutas normales.
- CaidaRapida: `objeto.getArea().y -= (velocidad * 1.55f) * delta`. Frutas podridas.
- CaidaZigZag: caída vertical + desplazamiento horizontal a 145 px/s con rebote real en bordes mediante `setDireccionHorizontal()`. Frutas power-up y dorada.

GM-5 Encapsulamiento y Principios de Orientación a Objetos

Encapsulamiento

Todos los atributos de todas las clases del dominio son privados. El acceso y modificación del estado se realizan exclusivamente mediante métodos públicos con validaciones. En PerroRecolector, el método daniar() incluye doble protección contra vidas negativas: primero verifica escudo activo, luego decrementa solo si vidas > 0 y finalmente garantiza que vidas no sea menor que 0. El método sumarPuntos() verifica si el doble puntaje está activo y aplica la multiplicación internamente, sin exponer ese detalle al exterior.

Herencia

Se aplica herencia en dos jerarquías: ObjetoCaida → {FrutaBuena, FrutaMala, FrutaEspecial, FrutaPowerUp} y Nivel → {NivelFacil, NivelDificil}. En ambas, la clase padre concentra estructura y comportamiento común y las subclasses solo redefinen lo que las distingue, evitando duplicación de código.

Polimorfismo

Lluvia almacena Array<ObjetoCaida> y llama fruta.aplicarEfecto(perro) sin conocer el tipo concreto en tiempo de compilación. GameLluvia referencia nivelActual como Nivel y lo reemplaza en tiempo de ejecución por NivelDificil al superar 200 puntos. FrutaBuilder devuelve ObjetoCaida independientemente del tipo construido.

Abstracción

ObjetoCaida abstrae el concepto de objeto que cae y tiene un efecto al ser recolectado. EstrategiaCaida abstrae el comportamiento de movimiento. GestorRecursos abstrae la carga, el ciclo de vida y la reproducción de todos los assets. TipoPowerUp abstrae los tipos de power-up disponibles mediante un enum, eliminando el uso de strings o enteros mágicos para identificarlos.

Open/Closed Principle (OCP)

En el proyecto este principio se aplica en la jerarquía de clases ObjetoCaida y en la interfaz EstrategiaCaida. La clase Lluvia trabaja con referencias de tipo ObjetoCaida y ejecuta el método aplicarEfecto() de forma polimórfica, sin depender de implementaciones concretas.

Gracias a este diseño es posible incorporar nuevas frutas o nuevos comportamientos de caída sin modificar las clases existentes. Por ejemplo, podría añadirse una nueva clase FrutaCongelante o una nueva estrategia de movimiento implementando EstrategiaCaida, manteniendo intacto el código actual. Esto permite extender la funcionalidad del sistema sin alterar componentes previamente desarrollados.

Liskov Substitution Principle (LSP)

Este principio se aplica en la jerarquía ObjetoCaida, donde FrutaBuena, FrutaMala, FrutaEspecial y FrutaPowerUp pueden ser almacenadas y utilizadas mediante referencias del tipo abstracto ObjetoCaida. La clase Lluvia administra una colección de objetos de tipo ObjetoCaida y ejecuta aplicarEfecto() sin necesidad de conocer el tipo concreto de fruta.

Así mismo, NivelFacil y NivelDificil pueden sustituir a la abstracción Nivel, permitiendo cambiar la dificultad durante la ejecución sin modificar la lógica del juego. Esto garantiza la correcta utilización del polimorfismo y la estabilidad del diseño orientado a objetos.

Dependency Inversion Principle (DIP)

En el proyecto este principio se observa mediante el uso de la interfaz EstrategiaCaida. La clase ObjetoCaida no depende directamente de las implementaciones concretas CaidaNormal, CaidaRapida o CaidaZigZag, sino de la abstracción EstrategiaCaida.

Las estrategias son asignadas durante la construcción de cada fruta mediante FrutaBuilder, permitiendo cambiar el comportamiento de movimiento sin modificar la clase ObjetoCaida. Gracias a ello se reduce el acoplamiento entre componentes y se facilita la incorporación de nuevas estrategias en futuras versiones del sistema.

GM-6 Patrón de Diseño Singleton

Singleton — GestorRecursos Gestión centralizada de assets

Situación identificada: Las texturas, sonidos y música de LibGDX son recursos costosos. Si varias clases los cargaran independientemente se duplicaría la memoria GPU y habría inconsistencias al liberar recursos con `dispose()`.

Contexto del proyecto: El juego necesita un único punto de acceso a los recursos gráficos y de audio, compartido por `GameLluvia`, `Lluvia`, `FrutaBuena`, `FrutaMala`, `FrutaEspecial` y `FrutaPowerUp`, entre otras clases.

Implementación realizada: `GestorRecursos` implementa Singleton con constructor privado y `getInstance()` estático. La primera llamada crea la instancia y carga todos los recursos (carga perezosa protegida por el flag 'cargado'). Provee métodos específicos como `reproducirSonidoRecolectar(0.75f)` y `reproducirSonidoDanio(0.75f)`. El método `destruir()` libera todo y resetea instancia a null.

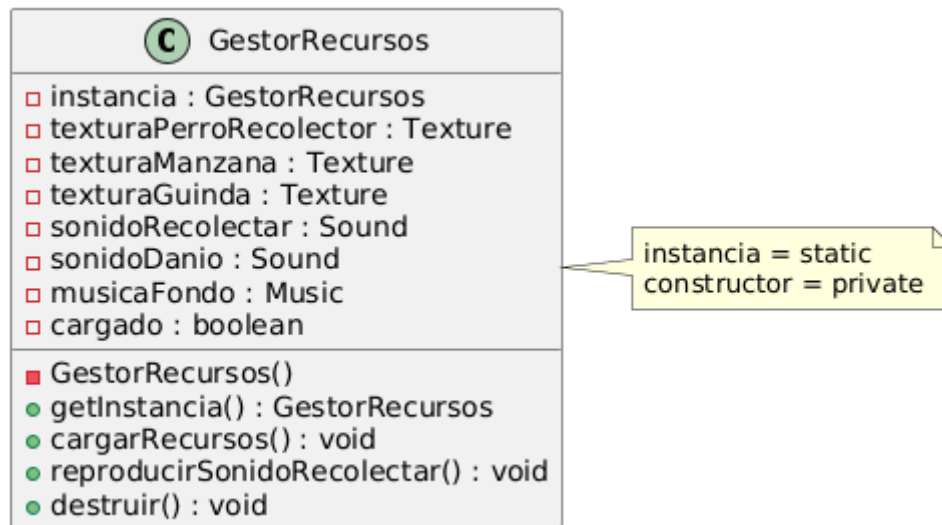
Singleton — GestorPuntaje Persistencia del puntaje máximo

Situación identificada: El puntaje máximo debe ser consistente y accesible desde menú, juego y final. Instancias múltiples podrían leer y escribir versiones distintas del dato persistente.

Contexto del proyecto: Se requiere un único estado global persistente entre sesiones usando las Preferencias de LibGDX, accesible desde múltiples pantallas sin acoplamiento directo.

Implementación realizada: `GestorPuntaje` implementa Singleton con carga perezosa de Preferences (se inicializan solo cuando `Gdx.app` está disponible). `guardarSiEsMayor()` compara antes de escribir para minimizar accesos a disco. El nombre de las preferencias es `'PerroRecolectorFrutalPuntajes'`.

Diagrama UML — Singleton



GM-7 Patrón de Diseño Strategy

Strategy — EstrategiaCaida Manejo flexible de distintos tipos de caída

Situación identificada: Distintos tipos de fruta deben caer de formas diferentes: recta normal, rápida o en zigzag con rebote. Implementar estos comportamientos con condicionales en ObjetoCaida viola el principio Open/Closed.

Contexto del proyecto: Al crear cada fruta se conoce qué tipo de caída tendrá. ObjetoCaida debe delegar el movimiento a un componente intercambiable sin conocer su implementación concreta.

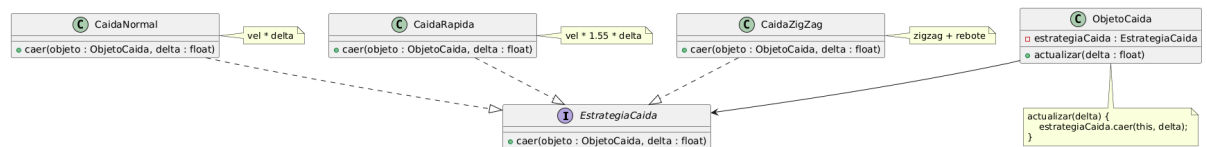
Implementación realizada: La interfaz EstrategiaCaida define caer(ObjetoCaida, float). ObjetoCaida la contiene como campo y delega actualizar() en ella. FrutaBuilder asigna la estrategia al construir: CaidaNormal para frutas normales, CaidaRapida para podridas, CaidaZigZag para power-ups y fruta dorada.

Implementaciones

- CaidaNormal: reduce y únicamente en función de velocidad * delta. Trayectoria vertical recta.

- **CaidaRapida:** multiplica la velocidad por 1.55f antes de aplicar el delta. Las frutas podridas caen 55% más rápido.
- **CaidaZigZag:** combina caída vertical con desplazamiento horizontal a 145 px/s. Al tocar el borde izquierdo o derecho llama setDireccionHorizontal() para invertir la dirección, produciendo rebote real en lugar de un seno matemático.

Diagrama UML — Strategy



GM-8 Patrón de Diseño Template Method

Template Method — Nivel Estructura común para la inicialización de niveles

Situación identificada: Todos los niveles deben inicializarse con el mismo proceso (nombre, velocidad, intervalo y probabilidades), pero con valores distintos. Duplicar ese proceso en cada subclase genera código repetido y frágil ante cambios.

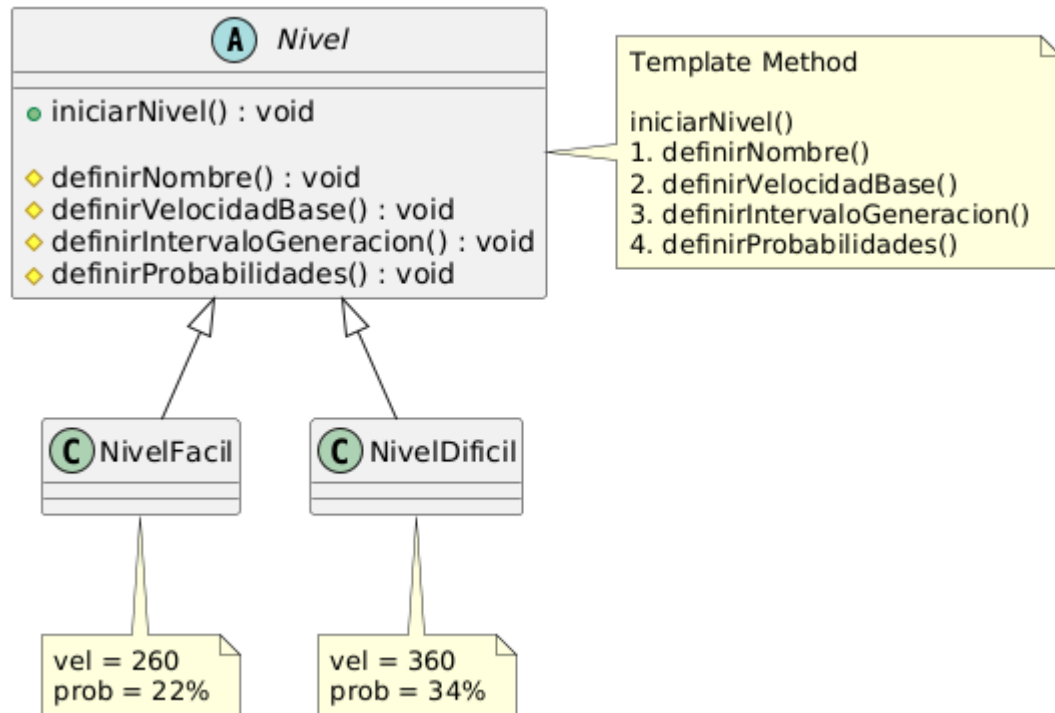
Contexto del proyecto: Se necesita garantizar que cualquier nivel futuro siga el mismo orden de inicialización, permitiendo personalizar los valores sin alterar el flujo.

Implementación realizada: Nivel define `iniciarNivel()` como método final que llama en orden a `definirNombre()`, `definirVelocidadBase()`, `definirIntervaloGeneracion()` y `definirProbabilidades()`, todos abstractos protegidos. `NivelFacil` y `NivelDificil` solo implementan esos cuatro métodos con sus valores específicos. El modificador final impide que las subclases alteren el flujo.

Clases Concretas

- `NivelFacil`: velocidad 260 px/s, intervalo 260 ms, probabilidad fruta mala 22%, especial 8%.
- `NivelDificil`: velocidad 360 px/s, intervalo 170 ms, probabilidad fruta mala 34%, especial 6%. Se activa automáticamente al superar 200 puntos.

Diagrama UML — Template Method



GM-9 Patrón de Diseño Builder

Builder — FrutaBuilder Construcción flexible de frutas

Situación identificada: Crear objetos ObjetoCaida requiere hasta 7 parámetros (textura, x, y, puntos, velocidad, estrategia, tipo y opcionalmente tipoPowerUp). Un constructor con tantos parámetros es ilegible y propenso a errores de orden.

Contexto del proyecto: En Lluvia.crearFruta() se construyen objetos con distintas combinaciones de parámetros según el tipo de fruta. Se necesita una forma legible y extensible de hacerlo sin exponer los constructores concretos.

Implementación realizada: FrutaBuilder implementa el patrón Builder con interfaz fluida: cada método con...() retorna this. El método construir() decide qué subclase instanciar según el tipo configurado (MALA, ESPECIAL, POWER_UP o BUENA por defecto). Para POWER_UP, usa el tipoPowerUp para construir FrutaPowerUp.

Ejemplo Real de Uso (Lluvia.crearFruta)

```

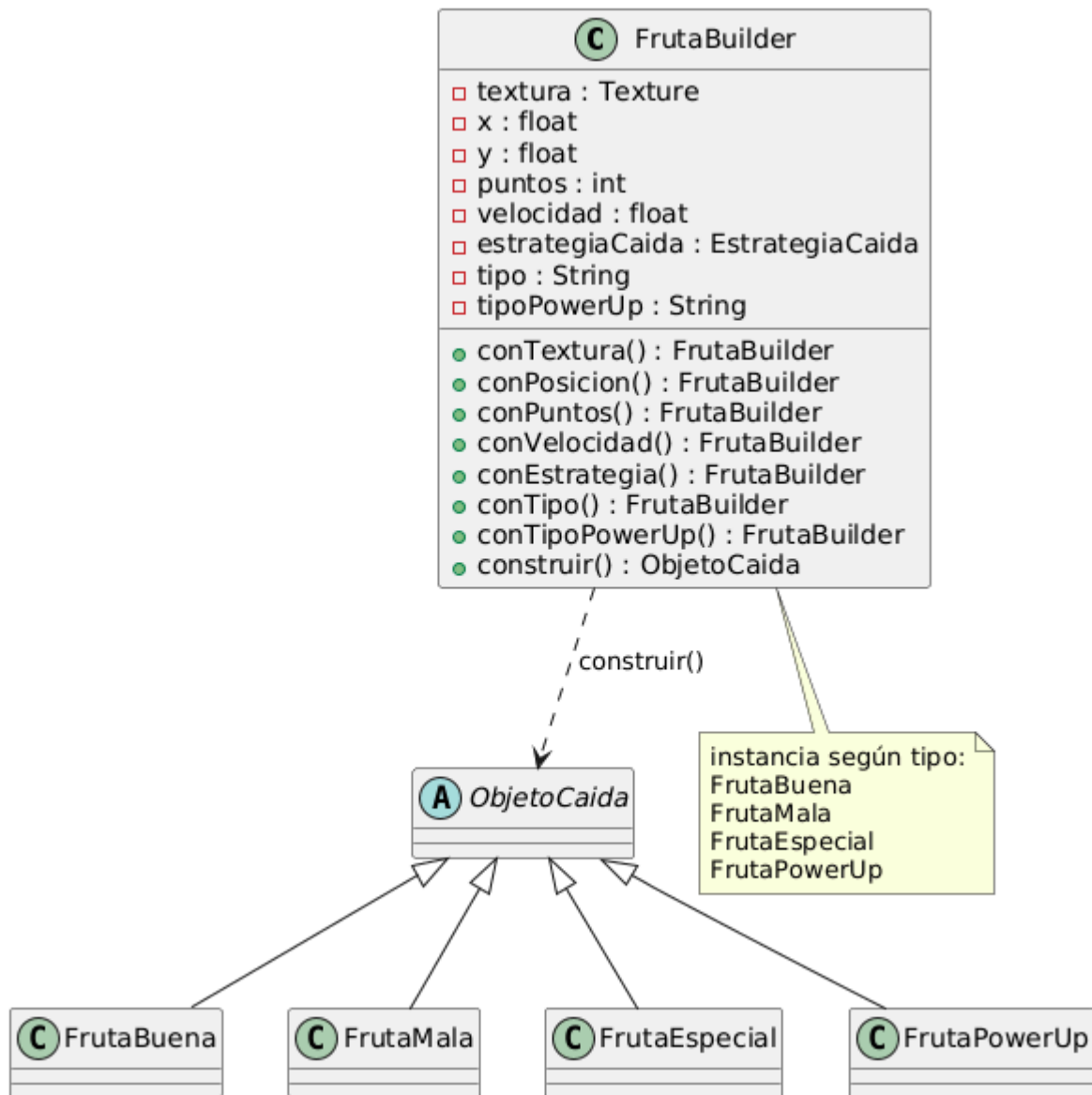
// Kiwi — power-up cámara lenta
ObjetoCaida fruta = new FrutaBuilder()
    .conTipo("POWER_UP")
    .conTipoPowerUp(TipoPowerUp.CAMARA_LENTA)
    .conTextura(GestorRecursos.getInstancia().getTexturaKiwi())
  
```

```

.conPosicion(MathUtils.random(0, 800 - 64), 480)
.conPuntos(0)
.conVelocidad(nivel.getVelocidadBase())
.conEstrategia(new CaidaZigZag())
.construir();

```

Diagrama UML — Builder



GM10: Integración final y gestión del repositorio del proyecto

https://github.com/ErosVerdejo/proyecto_juego_fruta

El proyecto fue gestionado con Git desde su etapa inicial, configurando el repositorio y organizando el flujo de control de versiones entre los integrantes del equipo. A lo largo del desarrollo se realizaron al menos 6 commits que documentan la evolución del proyecto.